Seranai

OpenFOS

Open Founder Operating System

Solution Design & MVP Scoping

24 Skills + 2 Reference Modules
Organized by FOUNDRY Framework

Seranai · March 2026

Created by Arshad Sayyad · A Seranai Product · Licensed CC BY-NC-SA 4.0

What Is This Skill?

The Solution Design & MVP Scoping skill is an AI-powered framework that helps you answer the question every first-time and repeat founder faces after validating their problem: What exactly do I build first?

It diagnoses which MVP failure mode your current plan is most at risk for, scores every feature using RICE and Kano prioritization, defines your MVP boundary with an explicit in-scope and deferred list, recommends a build vs. buy decision for each component, and produces a 30-day launch plan with weekly milestones and kill signals.

No spreadsheets, no engineering jargon, no guesswork. You describe your product and feature list, and the skill walks you through every scoping decision in a structured conversation.

THE CORE PRINCIPLE

Your MVP is not a product, it is an experiment.

Every feature that does not directly test your core hypothesis is borrowed time and borrowed money.
The skill scopes only what you need to learn, not everything you want to build.

By the end of one conversation (~20 minutes), you will have a diagnosed failure mode, a prioritized feature table with build/defer/cut decisions for each item, a locked MVP boundary, a component-level build vs. buy recommendation, a lightweight tech architecture matched to your product type, and a 30-day launch plan with defined kill signals.

Who Is This For?

This skill is designed for founders who have a validated problem and a product idea and are now deciding what to build first. It is especially useful if:

- You have a feature list and don't know what to cut
- You've been planning for weeks without shipping anything
- You keep adding features because they seem easy or obvious
- You're not sure what your MVP is actually designed to prove
- You want to know which components to build vs. buy or use off-the-shelf
- You're a first-time founder who has never scoped a product before
- You're a repeat founder who wants a structured pressure-test of your current plan

This skill works for any B2B product type: SaaS, marketplaces, AI-powered tools, internal tools, and more. Whether you are in Mumbai, Lagos, London, or São Paulo, the framework adapts to your team size, runway, and technology context.

How to Get Started

Using this skill requires three things: access to Claude (claude.ai or the Claude app), the Solution Design & MVP Scoping skill installed, and a description of your product or feature list in your own words.

Step 1: Open Claude and Start a New Conversation

Go to claude.ai or open the Claude app. Start a new conversation. That's it.

Step 2: Describe Your Product or Feature List

Tell Claude what you're building and who it's for. Share your current feature list if you have one, even a rough bullet list is enough. If you have outputs from earlier skills (validated problem, persona, value proposition), mention them or paste them in. The skill extracts what it needs automatically.

Examples of Great Starting Messages:

EXAMPLE 1: With a Feature List

"I'm building an HR compliance tool for SMBs in India. My current feature list is: automated compliance alerts, a document upload system, a reporting dashboard, a mobile app, an admin panel, and Slack notifications. I'm not sure what to cut for v1."

EXAMPLE 2: With a Problem Statement

"I've validated that logistics managers at mid-size freight companies waste 10+ hours a week on manual shipment tracking. I want to build something to fix this but I don't know where to start with the scope."

EXAMPLE 3: Minimal

"I'm building invoice management software for freelance designers. Help me scope what to build first."

All three will work. The more detail you provide, the sharper the initial diagnosis. But even a single sentence is enough, the skill will ask 3 to 5 targeted questions to fill in the gaps.

Step 3: Follow the Conversation

The skill guides you through four phases. You don't need to prepare anything in advance or remember any frameworks. Just respond naturally to the questions it asks. The whole process takes about 20 minutes.

What Happens: The Four Phases

The skill walks you through four phases, each building on the last. Here's what to expect at each stage.

PHASE 1: FAILURE MODE DIAGNOSIS *Identifying your biggest scoping risk before any decisions are made • ~3 minutes*	
What You Do	• Describe your product and current feature list (rough is fine) • Answer 3-5 sharpening questions about your hypothesis, customer, and build timeline
What You Get	• A diagnosis of which of the four MVP failure modes your plan is most at risk for, with a specific explanation of why • The single riskiest assumption your MVP needs to test, stated clearly • A confirmation of what the skill is carrying into the scoping phases

WHY THIS MATTERS:
Most scoping conversations jump straight to the feature list. This skill diagnoses the failure mode first because the right scoping decisions depend entirely on understanding what is most likely to go wrong with your current plan before you lock anything in.

PHASE 2: FEATURE PRIORITIZATION *RICE + Kano scoring for every feature • ~8 minutes*	
What You Do	• Walk through your feature list one item at a time • Provide estimates for Reach, Impact, Confidence, and Effort for each feature • Confirm the Kano category that best describes each feature (Must-Have, Performance, Delighter, Indifferent, or Reverse)
What You Get	• A complete feature priority table with a RICE score, Kano category, and a clear Build / Defer / Cut decision for each item • Flags for any features that fail the hypothesis filter with a one-line explanation of why • A ranked list of the features with the highest return on team effort

PHASE 3: MVP BOUNDARY + BUILD VS. BUY *Locking scope and making component decisions • ~5 minutes*	
What You Do	• Review the in-scope and deferred lists the skill produces • Confirm that the four lock conditions are met before scope is declared final • Review the build vs. buy recommendation for each component
What You Get	• A locked MVP boundary an explicit in-scope list and an explicit deferred list, each with a one-line rationale tied to the hypothesis • A scope confidence rating (Ship-Ready / Good with Trims / Over-Scoped / Rebuild Scope) • A component-level build vs. buy decision with recommended tools for everything you should buy • A lightweight tech architecture recommendation matched to your product type

PHASE 4: 30-DAY LAUNCH PLAN *Weekly milestones, kill signals, and scope creep protocol • ~4 minutes to read*	
What You Do	• Review the launch plan for accuracy against your team size and capacity • Confirm the kill signal, the specific observation that would tell you to stop and pivot • Confirm the success signal, the specific observation that would tell you to accelerate
What You Get	• A 30-day launch plan with weekly milestones tied to your specific MVP scope • A defined kill signal and a defined success signal stated before you build, not after • A scope creep response protocol, exactly what to do when someone asks to add a feature mid-build

Understanding Your Results

The Four MVP Failure Modes

Before any scoping decisions are made, the skill diagnoses which failure mode your current plan is most at risk for. Understanding your failure mode shapes every decision that follows.

Failure Mode	What It Looks Like	Why It Kills Products
Over-Scoping	Treating v1 like a full product launch. Too many features, too long to ship, too expensive to test.	By the time you ship, the market has moved, your assumptions are stale, and your team is exhausted.
Under-Hypothesis	No clear assumption being tested. Building without knowing what success looks like.	You ship, users arrive, and you have no idea whether the product is working or failing or why.
Complexity Creep	Simple core buried under edge cases, admin panels, and nice-to-haves that seemed easy to add.	Each small addition feels low-cost but collectively doubles build time and kills iteration speed.
Wrong Differentiator	Building generic commodity features instead of the one thing only you can do for this specific customer.	You ship a product indistinguishable from three existing tools and have no defensible reason to win.

The Feature Prioritization Framework: RICE + Kano

Every feature in your product plan is scored across two frameworks. Using both together eliminates the two most common prioritization mistakes: optimizing for effort efficiency while ignoring emotional response, or delighting users with features they don't actually need.

RICE Score: What Has the Highest Return on Effort?

The RICE framework answers one question: which features deliver the most value per unit of team effort? RICE Score = (Reach × Impact × Confidence) ÷ Effort. Higher scores mean more return per unit of team investment.

Factor	What It Measures	How to Score It
Reach	How many users encounter this feature per month?	Raw number of relevant users. not total users, only those who reach the feature.
Impact	How much value does it deliver per individual user?	3 = Massive, 2 = High, 1 = Medium, 0.5 = Low, 0.25 = Minimal.
Confidence	How certain are you of the Reach and Impact estimates?	100% = Data-backed. 80% = Mostly sure. 50% = Guessing.
Effort	How much total team work does this require?	Person-months across development, design, QA, and post-launch support.

Kano Category: How Will Customers Emotionally Respond?

The Kano model answers a different question: not how much value a feature delivers, but how customers will feel when it is present or absent. Combined with RICE, it prevents you from building high-effort features that users rationally value but emotionally ignore.

Category	Customer Response	MVP Decision Rule
Must-Have	Absence causes frustration. Presence is taken for granted and never praised.	Ship first the product is broken without it.
Performance	More of it = more satisfaction. Scales linearly. Users notice and compare.	Invest proportional to how much this differentiates you from alternatives.
Delighter	Unexpectedly. Not asked for. Creates disproportionate loyalty when discovered.	Include one great Delighter per MVP maximum. Never more.
Indifferent	Neither noticed when present nor missed when absent.	Cut immediately, it costs effort and generates nothing.
Reverse	A segment actively dislikes it when present.	Gate behind opt-in or remove entirely.

The Build vs. Buy Default Rule

The default is simple: buy or use no-code to validate anything that is not your core differentiator. Build only for the feature or mechanism that is uniquely yours, the thing no off-the-shelf tool can replicate without abandoning its own business model.

Component	Default Decision	Recommended Tools	Build Only If…
Authentication	Buy	Auth0, Clerk, Supabase Auth	Auth is your core IP e.g., biometric or custom identity federation.
Payments	Buy	Stripe, Paddle, Razorpay	Complex marketplace logic or split-payment rules are your primary differentiator.
CRM / Sales	Buy	HubSpot, Pipedrive, Attio	Customer relationship logic IS the product, not a support function.
Analytics	Buy	Mixpanel, Amplitude, PostHog	Data modeling or insight generation is your core value proposition.
Email / Notifications	Buy	SendGrid, Resend, Loops	A personalization or sequencing engine is your wedge against incumbents.
Internal workflows	Buy	Retool, Airtable, Zapier	Workflow automation itself is what you are selling.
AI features	Buy (API)	OpenAI, Anthropic, Mistral	You are training proprietary models on proprietary data as your moat.

Your core differentiator	Build	—	Always. This is the only thing worth building from scratch in v1.

Recommended Tech Stacks by Product Type

The skill recommends a specific stack based on your product type. The goal is to ship and learn in 60–90 days, not to build infrastructure that can serve 10 million users. Scale comes after validation, never before.

Product Type	Frontend	Backend	Hosting
SaaS Web App	React / Next.js	Node.js / FastAPI + PostgreSQL (Supabase)	Vercel / Railway
Mobile App	React Native / Flutter	Firebase / Supabase + Firestore	App Store / Play Store
AI-Powered Tool	Next.js	Python FastAPI + LLM API + Postgres + vector DB	Vercel + Modal
Internal Tool	Retool / Appsmith	REST API or direct DB connection	Self-hosted
Marketplace	Next.js	Node.js + PostgreSQL	Vercel + Railway

Tips for Getting the Most Out of This Skill

State your hypothesis before your feature list.

The single most important input is a clear, falsifiable hypothesis: “We believe [customer] will [do X] because [reason].” If you can’t state it in one sentence, the skill will help you find it — but bringing it in saves time and sharpens every scoring decision that follows.

“I don’t know” is a valid answer.

If you’re not sure how many users will reach a feature, say so. The skill will suggest a conservative default and flag it as an assumption. Uncertainty tells you what to measure first after you ship.

Be ruthless about Effort.

The most common RICE scoring mistake is underestimating Effort by ignoring QA, integrations, edge cases, and post-launch support. Every feature costs at least twice what you think it does in v1. When in doubt, double your estimate.

Never include more than one Delighter.

Delighters are high-risk in MVPs. They feel easy to justify (“it’ll only take two days”) but they dilute your core signal. If your MVP has two or more Delighters, cut one — the skill will flag this automatically.

The deferral list is as important as the build list.

A named deferral is a commitment not to build. Vague backlogs allow scope creep. When the skill produces a deferred list, keep it somewhere visible during the build. Every time someone asks to add a feature, check the deferral list first.

Define your kill signal before you build anything.

An MVP without a predetermined kill condition is an MVP that will never be declared a failure. Before any feature goes into v1 scope, agree: if we ship this and observe X, we stop and pivot. If we observe Y, we proceed. No ambiguous middle ground.

Best results come with prior skill outputs.

The skill works from a single product description, but the scoping is significantly sharper when it builds on a validated problem from Skill 1, a defined persona from Skill 4, and a value proposition from Skill 5. If you have those outputs, bring them into the conversation.

Frequently Asked Questions

Do I need a feature list to use this skill?
No. You can start with just a product description or a problem statement. The skill will help you generate an initial feature list and then prioritize it. But if you already have a list, even a rough one, bring it in. The prioritization will be faster and more specific.

What if I only have one or two features in mind?
That is a great starting point. The skill will help you confirm whether those features directly test your core hypothesis, identify any Must-Have features you may have overlooked, and check whether you're missing anything that would make the core flow work end-to-end.

What is the hypothesis filter and how does it work?
The hypothesis filter is a single question applied to every feature: does this feature directly test our core assumption? If you removed it, would you lose the ability to validate your key hypothesis? Yes means in scope. No means deferred to v2 regardless of how obvious or easy the feature seems.

What are the four lock conditions?
Before scope is declared locked, four conditions must all be true: (1) the core user flow works end-to-end, even if some steps are currently manual; (2) the primary value proposition is demonstrable to a real potential customer; (3) the team can collect the signal that proves or disproves the hypothesis; (4) no feature made it in because someone asked for it or because it seemed easy to add.

How is this different from just writing a product requirements document?
A PRD documents what you want to build. This skill decides what you should build first and why based on a scoring framework, a hypothesis filter, and explicit build vs. buy decisions. The output is a locked MVP boundary with rationale, not a backlog of requirements.

Created by Arshad Sayyad · A Seranai Product · Licensed CC BY-NC-SA 4.0

What if my product needs a lot of features to be useful at all?

Some products need a critical mass of features to deliver any value in a two-sided marketplace without both supply and demand mechanisms, for example. The skill accounts for this. Your MVP must be usable by five real target customers without your team manually intervening in the core flow. If it can't reach that bar, it is too thin. If it takes more than eight weeks to build, it is too thick.

Can I use this skill if I am non-technical?

Yes. The build vs. buy recommendations and tech stack suggestions are explained in plain language. You do not need to understand the technology to make the right decision. The skill explains what each tool does and why it is recommended for your specific product type.

How long does the full conversation take?

About 20 minutes for the full four-phase conversation, assuming you come in with a product description and a rough feature list. Faster if you have prior skill outputs. Longer if your feature list is extensive or your hypothesis needs to be developed from scratch.

The FOUNDRY Framework

FOUNDRY is the sequential journey framework that guides founders from initial idea through scale or exit. Each letter represents a stage, and each stage contains specific skills a founder needs to master.

Stage	Purpose	Skills
F – Foundation	Validate the idea. Confirm the problem is real, the market exists, and you know who to serve.	Skills 1–4
O – Outline	Design the business. Articulate value, map competition, model revenue, scope the MVP.	Skills 5–8
U – User	Acquire and understand customers. Discover needs, design GTM, set pricing, run experiments.	Skills 9–12
N – Nurture	Build relationships and traction. Craft the pitch, model financials, build the founder narrative.	Skills 13–15
D – Discipline	Execute and operate. Set OKRs, hire the team, handle legal and entity setup.	Skills 16–18
R – Results	Measure and iterate. Track KPIs, assess PMF, decide whether to pivot or persevere.	Skills 19–20
Y – Yield	Scale or exit. Confirm scaling readiness, plan liquidity events.	Skills 21–22

Cross-cutting skills (23–24) are not stage-specific and can be invoked at any point in the journey.

Reference modules (R1–R2) are shared evaluation lenses that multiple skills invoke. They are never used in isolation.

Created by Arshad Sayyad · A Seranai Product · Licensed CC BY-NC-SA 4.0